

Assignment 1: Neural Networks

In this assignment, you will be exploring factors that influence neural network performance and training time of neural networks on a supervised learning task. Supervised learning differs from reinforcement learning in that a set of correct answers are known in advance (as opposed to reinforcement learning, where the correct behavior is not necessarily known, just a reward function).

Learning Objectives

By the end of this assignment, students will have:

- Set up a reproducible machine learning environment,
- Implemented and trained a fully-connected neural network in PyTorch,
- Standardized training, evaluation, and testing data without leaking information from evaluation or testing data,
- Swept neural-network hyperparameters using seeded replicates,
- Interpreted learning curves and related hyperparameter choices to convergence, performance, and training time,
- Designed and carried out a short self-directed empirical investigation, and
- Written a short empirical report summarizing methods, evidence, and findings.

Computing Requirements

You are responsible for creating a Python environment in which your assignment runs. You may use a Python virtual environment, Conda environment, virtual machine, container, Windows Subsystem for Linux (WSL), or another suitable setup. Your environment must provide:

- a recent Python 3 version supported by the current PyTorch release,
- NumPy,
- PyTorch,
- Matplotlib, and
- Gymnasium with support for Atari, Box2D, classic-control, and other environments. Gymnasium is not required for the neural-network experiments in this assignment, but the same environment will be used in subsequent projects.

See the separate *Deep RL Computing Environment Setup Guide* for suggested platform-specific setup instructions. The assignment requirements above are the source of truth for the software your submitted code must use.

Data Provided: `swept_volume_data.npz`

You will be given a compressed numpy file called `swept_volume_data.npz` that stores several numpy arrays. (It will be posted on Canvas soon.) This file contains features (network inputs) and labels (expected network outputs) generated for a real world robotics problem. The problem involves estimating the volume of space passed through by a robot (composed of rigid bodies connected by rotational joints) moving between two configurations (lists of joint angle values). Predicting this volume (called a *swept volume*) is useful in planning robot motions in cluttered surroundings, as the probability of a motion resulting in a collision with something in the robot's environment is well estimated by the swept volume of that motion.

You may load `swept_volume_data.npz` into a dictionary and convert the numpy arrays to pytorch tensors with the following code:

```
import numpy as np
import torch

data = dict(np.load("swept_volume_data.npz"))
for key, value in data.items():
    print(f"{key}: {value.shape} shape, type {value.dtype}.")
    data[key] = torch.tensor(
        value,
        dtype=torch.float32)
```

Running the above code in the directory where `swept_volume_data.npz` is located will reveal the following data:

```
training_features: (100000, 14) shape, type float32.
training_labels: (100000, 1) shape, type float32.
evaluation_features: (10000, 14) shape, type float32.
evaluation_labels: (10000, 1) shape, type float32.
testing_features: (10000, 14) shape, type float32.
testing_labels: (10000, 1) shape, type float32.
```

Here, the first number of a shape is the number of rows of training (100,000), evaluation (10,000), or testing (10,000) data. The second number is the number of columns of floating point numbers for a given feature or label row. Here, features have 14 values, and labels have 1 value. The features here are two configurations. Features `[:, :7]` are the starting joint angles of the robot undergoing motion, while features `[:, 7:]` are the ending joint angles. Feature angles are given in radians. The label value associated with a feature is the volume passed through by the seven-jointed robot moving its pose between the starting and ending joint angles. Label values are Liters of swept volume.

You will be tasked with putting together code to train a fully connected neural network minimizing mean square error of output predictions (computed by the network from input features) relative to the associated labels. In this

assignment, the arrays whose names begin with `evaluation` serve as a validation set: they may be used during training and model selection. The testing arrays are held out for final evaluation.

Normalizing the Data

For this assignment, standardize both the input features and the swept-volume labels before training. For each feature dimension, compute the mean and standard deviation from the training examples used in that experimental condition, then transform that feature as

$$x_j^{\text{norm}} = \frac{x_j - \mu_j}{\sigma_j}.$$

Do the same for the swept-volume labels,

$$y^{\text{norm}} = \frac{y - \mu_y}{\sigma_y}.$$

The evaluation and testing sets must be transformed using *the mean and standard deviation from the corresponding training data*; do not compute new normalization statistics from evaluation or testing data.

For the CPU training-data-size experiment, the 1,000-example condition must compute its normalization statistics from those 1,000 training examples, and the 10,000-example condition from those 10,000 training examples. The GPU experiment uses the full training set, so its normalization statistics come from the full training set.

Use normalized mean square error as the optimization loss. For evaluation curves and final reported performance, convert predictions back to liters and report performance in the original units. Report both mean square error (MSE, in liters squared) and root mean square error (RMSE, in liters) for final test results.

Coding a Neural Network

You will need code for a fully connected neural network and training loop. Be sure you are working in the appropriate virtual environment (CPU or GPU).

You may use an AI coding assistant as allowed by the course AI policy. If you use one, a reasonable starting prompt is:

```
Generate a fully connected neural network with a variable number
of hidden layers and perform supervised training. Have training,
evaluation, and testing data. Use PyTorch.
```

You are responsible for understanding, running, debugging, and verifying any code produced with assistance. We will go over in class how to adapt a network and training loop to the features and labels from `swept_volume_data.npz`.

Training your Neural Network

Training data are used to adjust neural-network parameters. The evaluation data are a separate validation set used to monitor training and select model states and hyperparameters. The testing data are held out for final evaluation. **Do not use testing performance to choose a learning rate, architecture, training-data size, stopping point, or self-directed variant.**

Use `torch.optim.Adam` as the optimizer, with no weight decay, and use `torch.nn.MSELoss()` on the normalized labels as the training loss. Initialize linear-layer weights with `torch.nn.init.xavier_uniform_()` and biases with `torch.nn.init.zeros_()`.

The results of training depend on parameter initialization and minibatch ordering. Use the five replicate seeds

[0, 1, 2, 3, 4]

for every experimental condition. Set the seed before constructing and initializing the network for each replicate. If your code also uses NumPy randomness, seed NumPy consistently as well. Use the same five seeds for all conditions so that comparisons are reproducible.

Treat **one experimental run as one configuration with one seed**. Runs are independent and may be executed on different machines and combined afterward. For example, one CPU run is one choice of training-data size and learning rate with one of the five replicate seeds. This makes it possible to distribute the required sweep across multiple machines without changing the experimental design.

Training-time comparisons are only meaningful when the runs being compared are performed on the same hardware, or on machines with equivalent hardware. You may distribute performance experiments across machines, but use one machine (or equivalent hardware) for timing results that you compare in the report, and document the hardware used.

During each run, record evaluation performance every epoch and retain the model state from the epoch with the lowest evaluation error. A simple approach is to save `copy.deepcopy(model.state_dict())` whenever the evaluation error reaches a new minimum, then restore that state with `model.load_state_dict()` before final evaluation. Do not use the final epoch automatically if an earlier epoch performed better on the evaluation data.

If a required configuration diverges, produces NaNs, or otherwise fails to train successfully, record and report that behavior rather than silently changing the required hyperparameters.

For each required experimental condition, summarize the best evaluation RMSE and training time across the five seeds using the mean and standard deviation. Select the final configuration using *evaluation* results only. After the final configuration has been selected, evaluate the five retained model states for that configuration on the held-out testing set and report the mean and standard deviation of test MSE and test RMSE.

Assignment Choice: CPU or GPU Version

Choose between the CPU task or the GPU task below. In either case, you must use the normalization and training procedure above, document the resources and AI you use, comment your code, run the experiments yourself, and submit the complete runnable assignment on Canvas.

CPU: Training Data Size, Learning Rate Analysis

Create a neural network with one hidden layer of 64 neurons, input dimension 14, and output dimension 1. Use a training batch size of 1000 and the Adam optimizer. Investigate all combinations of learning rates

[1e-4, 1e-3, 1e-2, 1e-1]

and training-data sizes

[1000, 10000, 100000].

Use a fixed random ordering of the 100,000 training examples to define the 1,000-example and 10,000-example subsets, and reuse the same subsets for all five replicate seeds. The smaller subset should be contained in the larger subset. Compute normalization statistics separately from the training examples used in each data-size condition, as described above.

Train each of the 12 conditions for at least 1000 epochs using all five replicate seeds. For every run, record the evaluation RMSE by epoch, the best evaluation RMSE, the epoch at which it occurred, and the training time. Summarize the mean and standard deviation of best evaluation RMSE and training time for each condition. Include at least two evaluation-RMSE-versus-epoch plots that clearly illustrate the effects of learning rate and training-data size; plots should summarize the five replicates rather than showing only a single seed. A mean curve with a variability band (for example, mean $\pm$ one standard deviation across seeds) is one appropriate way to summarize the replicates.

Because an epoch processes the full training set, larger training sets require more minibatch updates and more computation per epoch. This is intentional in this assignment: consider that difference when discussing both performance and training time.

Choose the final CPU configuration using evaluation results only. Then evaluate the best retained model from each of the five seeds for that configuration on the full testing set and report mean and standard deviation of test MSE and RMSE.

GPU: Network Architecture Analysis

Create networks with hidden-layer counts

[1, 2, 3]

and neurons per hidden layer

[32, 64, 128, 256].

Use learning rate $1\text{e-}3$, the full training data set, a training batch size of 10000, and the Adam optimizer. Use the normalization statistics from the full training set.

Train each of the 12 architecture conditions for at least 1000 epochs using all five replicate seeds. For every run, record evaluation RMSE by epoch, best evaluation RMSE, the epoch at which it occurred, and training time. Summarize the mean and standard deviation of best evaluation RMSE and training time for each condition. Include at least two evaluation-RMSE-versus-epoch plots that clearly illustrate the effects of network depth and width; plots should summarize the five replicates rather than showing only a single seed. A mean curve with a variability band (for example, mean $\pm$ one standard deviation across seeds) is one appropriate way to summarize the replicates.

Choose the final GPU configuration using evaluation results only. Then evaluate the best retained model from each of the five seeds for that configuration on the full testing set and report mean and standard deviation of test MSE and RMSE.

GPU operations may execute asynchronously. If you time GPU training, call `torch.cuda.synchronize()` immediately before starting and immediately before stopping the timer so that timing measurements include the GPU work being measured.

Self-Directed Investigation (491 and 591)

Now that you have completed the required sweep, choose one change to the architecture or training procedure that you have not already investigated. State a hypothesis about what the change will do and why. Compare an appropriate baseline with at least one new experimental condition using the same five replicate seeds as the main assignment. Use evaluation results to compare the conditions and explain whether your hypothesis was supported. Once you have fixed the final variant you intend to report, you may evaluate that final variant on the testing set.

Possible changes include, but are not limited to, another network width or depth, a different batch size, a different activation function, regularization, a learning-rate schedule, or another training choice that you can motivate. The goal is not simply to find a lower error; the goal is to make a prediction, gather evidence, and explain the observed behavior.

591 Graduate Extension: Follow-up Investigation

Graduate students will extend the self-directed investigation to examine *why* an observed effect occurs. Select an interesting result from either the required experiments or your self-directed experiment and propose an explanation for the observed behavior. Design additional experiments that test that explanation.

The 591 extension must:

- include at least **three additional experimental conditions** that are not already required elsewhere in the assignment,
- use the same five replicate seeds as the main experiments,
- vary factors in a way that helps distinguish between plausible explanations,
- include at least **two additional diagnostic plots**, and
- discuss whether the additional evidence supports the original explanation and what alternative explanations remain.

Appropriate follow-up analyses might compare training and evaluation error to investigate overfitting, examine additional values around an interesting learning rate or architecture, compare model capacity with performance, plot predicted versus true swept volume or prediction residuals, or test a training choice that could explain an observed convergence difference. You are not required to use any particular advanced neural-network technique; choose experiments that help answer the question raised by your results.

Use of AI and Other Resources

AI tools may be used for programming, plotting, and writing assistance as allowed by the course AI policy. You must run the experiments yourself, analyze the results yourself, and draft your interpretation of the results before using AI to revise or improve that interpretation. You are responsible for understanding and verifying submitted code, plots, numerical results, and written claims. Document all AI tools and other outside resources used in the acknowledgment section of your report.

Things you need to do before Turn In

- **491 only:** Form a group of two to work on the project together.
- **591 only:** Work individually and complete the graduate follow-up investigation described above.
- Create a working Python environment that satisfies the Computing Requirements above. You may use any suitable platform or environment-management approach.
- Make sure your submitted code uses relative paths. It must run after `swept_volume_data.npz` is placed in the extracted submission directory; do not hard-code paths specific to your account or computer.
- Do not include your virtual-environment directory or `swept_volume_data.npz` in the submission ZIP.

Implementation Requirements

Your submitted implementation must contain a file named `neural_network.py`. It may import additional Python files that you include in your submission. `neural_network.py` must contain a class `NeuralNetwork` implementing a fully connected neural network with:

- an `__init__()` method with four integer parameters:
 - `in_dimension`, the feature size,
 - `out_dimension`, the label size,
 - `hidden_layers`, the number of hidden layers, and
 - `neurons_per_hidden_layer`, the neurons per hidden layer;
- linear-layer weights initialized using `torch.nn.init.xavier_uniform_()` and biases initialized using `torch.nn.init.zeros_()`; and
- a `forward()` method that takes a single parameter `x` and passes it through the network, with `torch.nn.ReLU()` activation after every layer except the last.

The training program must support running **one configuration and one replicate seed at a time**. Command-line arguments (or an equivalent documented interface) must be sufficient to specify the training-data size, learning rate, architecture, batch size, replicate seed, number of epochs, and device needed for a run. Independent runs may be launched on different machines.

Each run must save enough information to reproduce and analyze that run, including:

- the complete experiment configuration and seed,
- per-epoch training and evaluation history,
- the best evaluation epoch and best evaluation performance,
- training time, and
- the best retained model checkpoint together with the normalization statistics required to use that checkpoint later.

Output directories or filenames must uniquely identify the run configuration and seed. Your program should prevent accidental overwriting of an existing run unless the user explicitly chooses to replace it.

The program must implement the normalization, training, evaluation, model-selection, and result-recording procedures required above. It must also provide a quick verification mode that can be run with:

```
python3 neural_network.py --quick-test
```

The quick-test mode must load the data, compute and apply training-derived normalization, create a network, perform a short training run on a small subset, run the *evaluation* pipeline, and exit successfully without running the full assignment. **Quick-test mode must not report performance on the held-out testing set.** This mode is only for checking that the implementation is runnable; it does not replace the required experiments.

Your README must state:

- the exact command needed to run one experimental configuration and seed,
- the exact quick-test command,
- how to aggregate the independent runs into the complete sweep used in your report, and
- how to perform final held-out testing after the final configuration has been selected.

A grader should not need to edit your source code to change paths, launch a run, aggregate results, or reproduce the submitted analysis.

What to Turn In on Canvas

Submit **two files** to Canvas. Canvas is the final submission location.

1. `NeuralNetworkReport.pdf`

Your report must include:

- **Names and assignment choice:** list your name(s), course number (491 or 591), and whether you completed the CPU or GPU assignment.
- **Introduction:** describe the swept-volume dataset and the questions about supervised neural-network learning that the assignment investigates.
- **Methodology:** describe the network, normalization procedure, optimizer, loss, five replicate seeds, model-selection procedure, and how evaluation and testing data were used.
- **Required Results and Discussion:**
 - CPU assignment: discuss the effects of learning rate and training-data size on convergence, evaluation performance, and training time.
 - GPU assignment: discuss the effects of network depth and width on convergence, evaluation performance, and training time.
 - Include at least two evaluation-RMSE-versus-epoch plots for the required sweep.

- Include a table giving mean and standard deviation of best evaluation RMSE and training time for every required condition.
- Identify the final configuration selected using evaluation data and report mean and standard deviation of test MSE and RMSE across its five retained models.
- **Self-Directed Results and Discussion:** state your hypothesis, describe the baseline and new condition(s), present the relevant evaluation evidence and at least one plot, and explain why the change did or did not have the expected effect. Report final test performance only for the variant fixed before looking at its test result.
- **591 Follow-up Analysis (591 only):** present the three or more additional conditions and at least two additional diagnostic plots. Explain how the experiments test your proposed explanation and what conclusions are supported or not supported by the evidence.
- **Acknowledgment:** document all AI tools and other outside resources used in coding, plotting, analysis, or preparation of the report. For AI tools, include the tool name and model or engine when known, the prompts or commands used, the purpose of each interaction, what output was incorporated into the assignment, and how you verified the resulting code, plots, numerical results, analysis, or writing.

2. NeuralNetworkSubmission.zip

The ZIP must contain a complete runnable snapshot of the work used to produce the report. Include:

- `neural_network.py` and **all student-written source files** used to train models, aggregate independent runs, perform final testing, and generate plots for the report;
- `README.md`, containing:
 - name(s), course number, and CPU/GPU assignment,
 - Python and PyTorch versions used,
 - the exact command to run one experiment (one configuration and one seed),
 - the exact quick-test command,
 - instructions for aggregating independent runs into the complete sweep,
 - instructions for performing final held-out testing after configuration selection, and
 - a short description of the output files produced by the code;
- `requirements.txt`, listing the Python packages required to run the submitted code;

- a `runs/` directory (or equivalent clearly documented organization) containing the saved output from every required and self-directed experimental run. Each run should include its configuration/seed summary, per-epoch history, and best retained model checkpoint;
- an aggregated `results.csv`, with one row per experimental run and enough information to identify the experiment, condition, seed, best epoch, best evaluation performance, training time, and any final test metrics that were computed;
- the per-epoch history data used to generate the learning-curve plots for all required and self-directed experiments (these may be stored separately for each run, for example as `history.npz`);
- a `plots/` directory containing the plots used in the report; and
- any additional small configuration or data-summary files required by your program.

Do **not** include your virtual environment, large package caches, or `swept_volume_data.npz`. The grader will provide the dataset when testing your submission.